

NAME :- Arvind S

REG NO :- 192B25192

COURSE :- Compiler Design

Experiment 1:- Identify Invalid C Identifier using LEX

=> Aim:- To write a LEX program to identify invalid identifiers from the given input based on the rules of C identifiers.

=> Algorithm :-

1. Start the LEX program
2. Read one identifier at a time
3. Check whether the first character is a letter or underscore.
4. Check whether the remaining characters are letters, digits, or underscores.
5. If all conditions are satisfied, print the identifier as valid.
6. Otherwise, print it as invalid
7. Stop.

=> Program :-

```
%{
```

```
#include <stdio.h>
```

```
%}
```

```
%*%
```

```
^[a-zA-Z_] [a-zA-Z0-9_]* $
```

```
printf("valid\n");
```

```
*
```

```
printf("Invalid\n");
```

```
%.*%
```

```
int main ()
```

```
{
```

```
yy lex ();
```

```
return 0;
```

```
}
```

```
int yy corap ()
```

```
{
```

```
return 1;
```

```
};
```

=> Output

total

valid Identifier

2 value

Invalid Identifier

=> Result

Thus the LEX program successfully identifies valid and Invalid C identifiers.

Experiment 8: Lex Preprocessor Directives.

Aim:- To identify and print preprocessor directives from a C program.

Algorithm:-

1. Read the C source program
2. Search for #include and #define
3. Print the matching lines.

Program:-

```
% {  
    #include <stdio.h>  
  
    %}  
  
%%  
^ [\t]* # [\t]* include.*  
{ printf (" %s\n", yytext); }  
  
^ [\t]* # [\t]* define.*  
{ printf (" %s\n", yytext); }  
  
.\n  
%%  
  
int main()  
{  
    yyclex();  
    return 0;  
}  
int yywrap()  
    { return 1;  
}
```

→

g

include <string>

=> Output :-

include <stdio.h>

define MAX 100

=> Result :-

Thus the LEX program successfully identifies and prints preprocessor directives.

→ Experiment 3 :- Recursive Descent parser

Aim :- TO implement a Recursive descent parser for the given grammar.

Grammar :-

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow id$

=> Algorithm :-

1. Read the input
2. Check for id.
3. Check for zero or more +id.
4. If the complete string matches, accept it.
5. Otherwise, reject it.

Program:-

```
#include <stdio.h>
#include <string.h>
```

```
char s[50];
```

```
int i = 0;
```

```
int T()
```

```
{
```

```
    if (s[i] == 't' &&
```

```
        s[i+1] == 'd')
```

```
    {
```

```
        i += 2;
```

```
        return 1;
```

```
    }
```

```
    return 0;
```

```
}
```

```
int E()
```

```
{
```

```
    if (!T()) return 0;
```

```
    while (s[i] == 't')
```

```
    {
```

```
        i++;
```

```
        if (!T()) return 0;
```

```
    }
```

```
    return 1;
```

```
}
```

```
int main()
```

```
{
```

```
    printf("Enter string:");
```

```
    scanf("%s", s);
```

=> Program

```
if (E() && s[i] == '\0')  
    printf("Accepted\n");  
else  
    printf("Rejected\n");  
  
return 0;  
}
```

=> Output

Enter string:- id + id + id
Accepted.

=> Result:-

Thus, the Recursive descent parser successfully checks the input string according to the given grammar.

Experiment 4: Three Address code.

Aim:-

To generate three address code for an arithmetic expression.

Algorithm:-

1. Read the expression.
2. Identify operators and operands.
3. perform higher - precedence operations first
4. Generate temporary variables
5. Display TAC.

→ Program :-

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    char a, b, c, d;
```

```
    printf ("Enter : ");
```

```
    scanf ("%c = %c + %c * %c", &a, &b, &c, &d);
```

```
    printf ("t1 = %c * %c\n", c, d);
```

```
    printf ("%c = %c + t1\n", a, b);
```

```
    return 0;
```

```
}
```

→ Output

Enter : a = b + c * d

t1 = c * d

a = b + t1

⇒ Results:-

Thus, the C program successfully generates three Address code for the given arithmetic expression.